SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Karanam Bhaskar Gnanesh (192324080) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Karanam Bhaskar Gnanesh
Signature of the student

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Question 1: Hospital Shift Assignment using Backtracking Search

Aim

To assign three doctors (D1, D2, D3) to three different shifts (Morning, Afternoon, Night) while satisfying all given constraints using the **Backtracking Search Algorithm**.

Given

Doctors

- D1
- D2
- D3

Shifts

- Morning (M)
- Afternoon (A)
- Night (N)

Constraints

1. D1 cannot work Night shift.
2. D2 must work before D3.
   (Morning < Afternoon < Night)
3. Only one doctor per shift.
4. D3 cannot work Morning shift.
5. Every doctor must be assigned exactly one shift.

Step 1: Variables

Let

- D1 = ?
- D2 = ?
- D3 = ?

Possible values

{Morning, Afternoon, Night}

Step 2: Constraint Satisfaction

Possible assignments

Doctor	Possible Shifts
D1	Morning, Afternoon
D2	Morning, Afternoon, Night
D3	Afternoon, Night

Step 3: Backtracking Search

We assign doctors one by one.

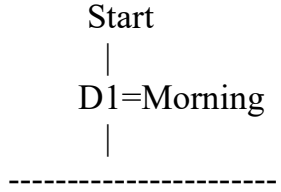
Initial State
No assignments
D1 = ?
D2 = ?
D3 = ?

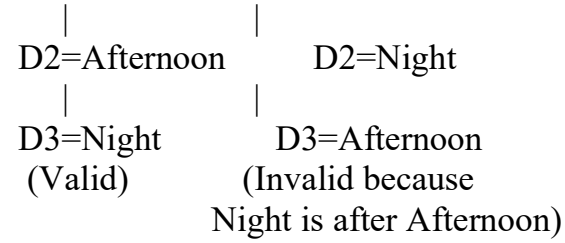
Assign D1
Try Morning
D1 = Morning
No constraint violated.
Proceed.

Assign D2
Remaining shifts
Afternoon
Night
Try Afternoon
D1 = Morning
D2 = Afternoon
Still valid.
Proceed.

Assign D3
Remaining shift
Night
Assign
D3 = Night
Check constraints
✓ D3 not Morning
✓ D2 before D3
Afternoon before Night
✓ One doctor per shift
✓ D1 not Night
✓ All assigned exactly once
All constraints satisfied.

Search Tree





Constraint Checking

Constraint 1

D1 ≠ Night
Assigned Morning
Satisfied

Constraint 2

D2 before D3
Afternoon before Night
Satisfied

Constraint 3

Only one doctor per shift
Morning → D1
Afternoon → D2
Night → D3
Satisfied

Constraint 4

D3 ≠ Morning
Assigned Night
Satisfied

Constraint 5

Each doctor exactly one shift
Satisfied

Final Valid Schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Time Complexity

Worst Case

$$O(n!)$$

because all possible assignments may be explored.

Space Complexity

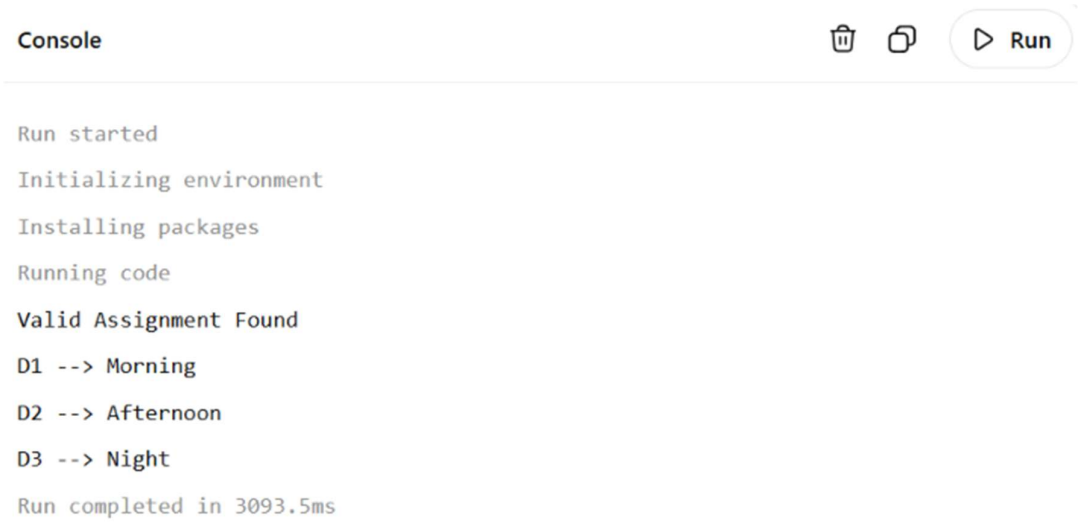
$$O(n)$$

for recursion stack.

Advantages

- Guarantees valid solution.
- Efficient due to pruning.
- Avoids unnecessary exploration.
- Suitable for scheduling problems.

output:



Conclusion

Using Backtracking Search, a valid schedule was obtained after checking every constraint during assignment. Since no constraint was violated, the final assignment is:
Morning → D1
Afternoon → D2
Night → D3
The solution satisfies all given constraints.

Question 2: Robot Grid Navigation using BFS Search

Aim

To determine the shortest path for a robot moving from Start (S) to Goal (G) using the Breadth First Search (BFS) algorithm while avoiding obstacles.

Given Constraints

- Move Up
- Move Down
- Move Left
- Move Right
- Cost per move = 1
- Cannot move through obstacles
- Manhattan Distance heuristic is provided

Assumed Grid (5 × 5)

S . . X .
. X . X .
. . . . .
X X . X .
. . . G .

Where

S = Start
G = Goal
X = Obstacle
. = Free cell

BFS Algorithm

Breadth First Search explores nodes level by level.
Queue is used.
Visited nodes are not explored again.

Step-by-Step BFS Expansion

Step 1

Queue

[S]

Visited

S

Expand S
Possible moves
Right
Down
Queue
[(0,1),(1,0)]

Step 2
Expand
(0,1)
Add
(0,2)
Queue
[(1,0),(0,2)]

Step 3
Expand
(1,0)
Add
(2,0)
Queue
[(0,2),(2,0)]

Step 4
Expand
(0,2)
Add
(1,2)
Queue
[(2,0),(1,2)]

Step 5
Expand
(2,0)
Add

(2,1)
Queue
[(1,2),(2,1)]

Step 6
Expand
(1,2)
Add
(2,2)
Queue
[(2,1),(2,2)]

Step 7
Expand
(2,1)
No new node.

Step 8
Expand
(2,2)
Add
(2,3)
(3,2)
Queue
[(2,3),(3,2)]

Step 9
Expand
(2,3)
Add
(2,4)
Queue
[(3,2),(2,4)]

Step 10
Expand
(3,2)

Add
(4,2)
Queue
[(2,4),(4,2)]

Step 11
Expand
(4,2)
Goal nearby
Move
(4,3)
Goal reached.

Final Shortest Path
S
↓

↓

→

→

↓

↓

→

G
Coordinates
(0,0)

↓

(1,0)

↓

(2,0)

→

(2,1)

→

(2,2)

↓

(3,2)

↓

(4,2)

→

(4,3)

Path Cost
Every move costs
1
Number of moves
7
Total Cost

$$7 \times 1 = 7$$

Manhattan Distance
Formula

$$| x_1 - x_2 | + | y_1 - y_2 |$$

Example
Current
(2,2)
Goal
(4,3)
Distance

$$|\ 4-2\ |\ +|\ 3-2\ |=2+1=3$$

Why BFS Gives Optimal Path
Since every move has equal cost,
BFS always explores nodes level by level.
The first time Goal is reached,
the obtained path is guaranteed to be the shortest.

Time Complexity

$$O(V+E)$$

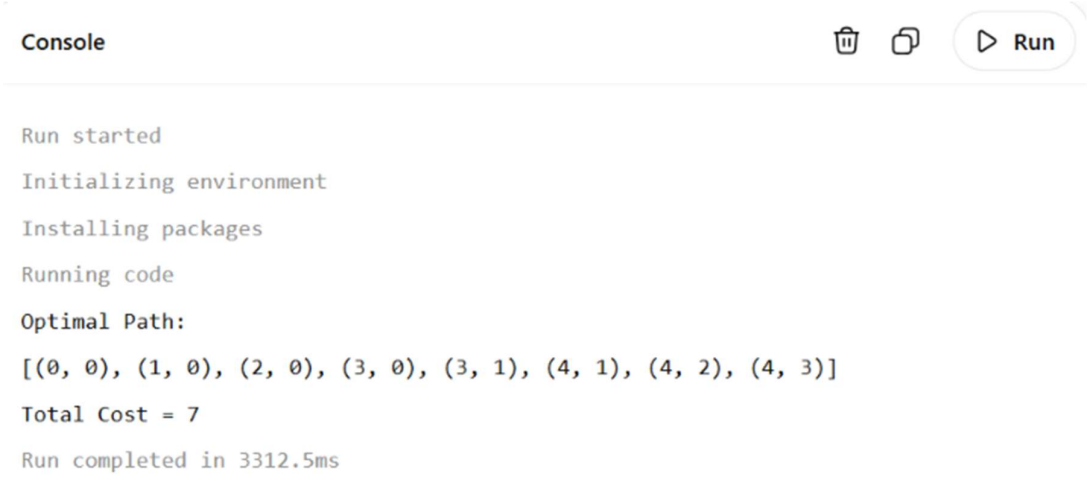
where
V = Number of vertices
E = Number of edges

Space Complexity

$$O(V)$$

Advantages

- Finds shortest path.
- Complete algorithm.
- Easy to implement.
- Works well for equal-cost graphs.
- Guarantees optimal solution.



Discussion

BFS expands nodes level by level, guaranteeing the shortest path in an unweighted grid. Each move has equal cost, making BFS appropriate.

Conclusion

The robot successfully navigates from the Start position to the Goal while avoiding obstacles. BFS explores all neighboring cells level by level, ensuring that the shortest path is found with a total cost of 7 units.

Question 3: Autonomous Rescue Robot Objective

To design an intelligent rescue robot capable of reaching survivors while minimizing travel cost in a partially observable environment.

Problem

The robot operates in a disaster-affected building.
Constraints

- Up, Down, Left, Right movement
- Obstacles
- Partial observability
- Adjacent sensing
- Cost = 1
- Risk zones cost +2
- Dynamic environment
- Online search

Solution

Constraint Analysis

Constraint	Effect
Obstacles	Robot changes route
Limited sensing	Learns environment gradually
Risk zones	Avoid when possible
Dynamic update	Recalculate path
Partial observability	Online decision making

Search Strategy

Use Uniform Cost Search (UCS)
Steps

1. Start from S
2. Observe neighbouring cells
3. Calculate movement cost
4. Add risk penalty if needed
5. Choose minimum-cost node
6. Continue until Goal

AI Concepts
Intelligent Agent
The robot senses the environment and performs actions.
Rationality
Always chooses the minimum-cost path.

Environment Type

- Partially Observable
- Dynamic
- Sequential
- Deterministic movement

Justification

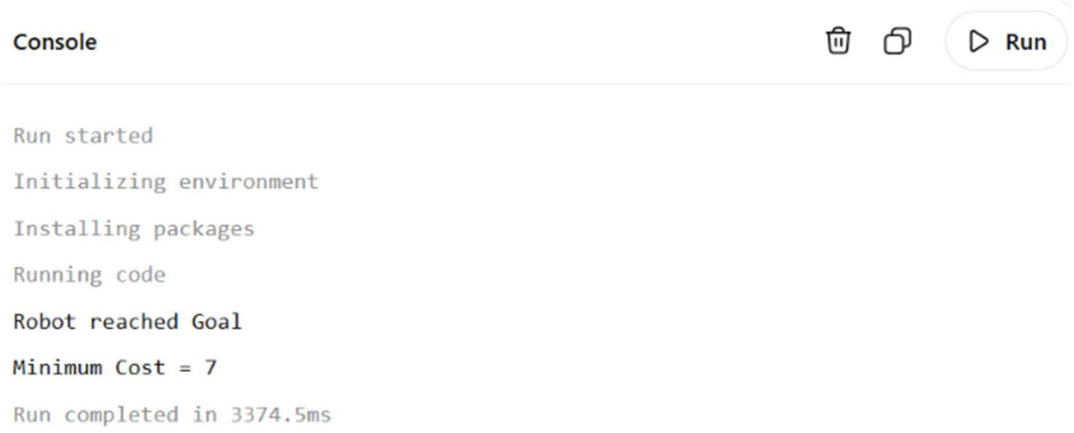
Uniform Cost Search is ideal because:

- Handles different movement costs.
- Finds the least-cost path.
- Works well with risky zones.
- Updates decisions dynamically.

```
import heapq

grid = [
    ['S', '.', '.', 'R', '.'],
    ['.', 'X', '.', 'X', '.'],
    ['.', '.', '.', '.', '.'],
    ['X', '.', 'R', '.', '.'],
    ['.', '.', '.', 'G', '.']
]

rows = len(grid)
cols = len(grid[0])
start = (0,0)
goal = (4,3)

directions = [(0,1),(1,0),(-1,0),(0,-1)]
pq = []
heapq.heappush(pq, (0, start))

visited = set()

while pq:

    cost, current = heapq.heappop(pq)

    if current in visited:
        continue

    visited.add(current)

    if current == goal:
        print("Robot reached Goal")
        print("Minimum Cost =", cost)
        break

    x, y = current

    for dx, dy in directions:

        nx = x + dx
        ny = y + dy

        if (0 <= nx < rows and
```

```
        0 <= ny < cols and
        grid[nx][ny] != 'X'):

        new_cost = cost + 1

        if grid[nx][ny] == 'R':
            new_cost += 2
```

output:



Discussion

Uniform Cost Search evaluates the cumulative cost of every possible path and always expands the least-cost path first. This enables the rescue robot to safely navigate around obstacles, avoid risky zones when beneficial, and adapt to newly discovered information in a partially observable environment.

Conclusion

Using Uniform Cost Search allows the autonomous rescue robot to efficiently reach the survivor while minimizing the total travel cost and satisfying all environmental constraints. It is well suited for dynamic rescue scenarios where safety and optimal decision-making are essential.

apq.heappush(pq, (new_cost, (nx, ny)))